



操作系统课程实验

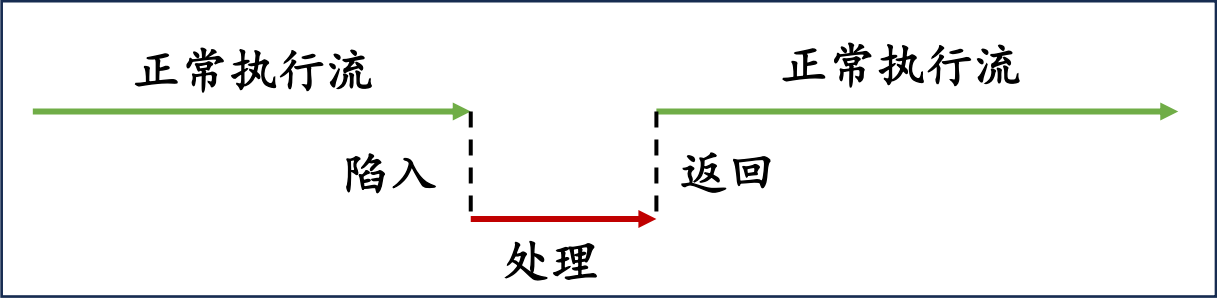
——从0到1的小型内核实现

2025年 秋季学期

lab-3 中断异常初步

1. 初步认识中断和异常
2. 串口中断
3. 时钟中断
4. 实验小结

1. 初步认识中断和异常



中断和异常是RISC-V定义的重要概念，被统称为陷阱
陷阱？好奇怪的名字.....

生活中的例子：小熊在一条林间小路上行走，不小心掉进一个大坑里，昏迷一段时间后，爬了出来，继续向前——陷阱全流程（陷入、处理、返回）

简单来说，陷阱是对程序正常执行流的临时性打断

	中断	异常
打断类型	可预计 / 所期待的打断	不可预计 / 并不期待的打断
举例说明	屏幕每隔30分钟提醒你休息1分钟	水果削到一半突然割伤手指
返回位置	PC + 4（执行下一条指令）	PC（重新执行当前指令）
具体种类	时钟中断、软件中断、外设中断	内存读写越界、非法指令等

1. 初步认识中断和异常

Interrupt	Exception Code	Description
1	0	User software interrupt
1	1	Supervisor software interrupt
1	2	<i>Reserved for future standard use</i>
1	3	Machine software interrupt
1	4	User timer interrupt
1	5	Supervisor timer interrupt
1	6	<i>Reserved for future standard use</i>
1	7	Machine timer interrupt
1	8	User external interrupt
1	9	Supervisor external interrupt
1	10	<i>Reserved for future standard use</i>
1	11	Machine external interrupt
1	12-15	<i>Reserved for future standard use</i>
1	≥16	<i>Reserved for platform use</i>
0	0	Instruction address misaligned
0	1	Instruction access fault
0	2	Illegal instruction
0	3	Breakpoint
0	4	Load address misaligned
0	5	Load access fault
0	6	Store/AMO address misaligned
0	7	Store/AMO access fault
0	8	Environment call from U-mode
0	9	Environment call from S-mode
0	10	<i>Reserved</i>
0	11	Environment call from M-mode
0	12	Instruction page fault
0	13	Load page fault
0	14	<i>Reserved for future standard use</i>
0	15	Store/AMO page fault
0	16-23	<i>Reserved for future standard use</i>
0	24-31	<i>Reserved for custom use</i>
0	32-47	<i>Reserved for future standard use</i>
0	48-63	<i>Reserved for custom use</i>
0	≥64	<i>Reserved for future standard use</i>

左边的表格显示了RISC-V支持的所有中断和异常的具体类型

U-mode S-mode M-mode 各有以下三种中断：

- 时钟中断：每隔若干时钟周期触发一次 (系统心跳)
- 软件中断：程序员通过寄存器操作触发 (可扩展性)
- 外设中断：外部设备发出中断信号触发 (外设支持)

异常通常不区分特权级且种类更多，大致分为以下三类：

- 系统调用：U-mode S-mode M-mode的环境调用 (可控)
- 缺页异常：指令执行、数据读、数据写遇到的page fault (可控)
- 其他异常：非法指令、访问不对齐等 (不可控)

lab-3 只会涉及三种中断的处理，不涉及异常处理

之后的实验会涉及更多中断和异常类型，逐步完善陷阱处理系统

2. 串口中断

串口的输入输出模式是不同的：

- 输出：主执行流中的一个函数调用（主动）
- 输入：等待用户在某个时刻敲击键盘（被动）

OS内核在主执行流中通过while循环来等待这种不知道何时发生的外部输入是**不经济的**，需要“中断机制”在事件发生时主动告知内核，安全地打断主执行流，事件处理完毕后恢复主执行流。

因此，实现串口中断的核心就是解决以下两个问题：

- 如何**告知**内核发生了串口中断？
- 如何完成主执行流和陷阱处理执行流的**安全切换**？

忙等模式

每隔5秒刷新一次网页
查看期末成绩是否发布

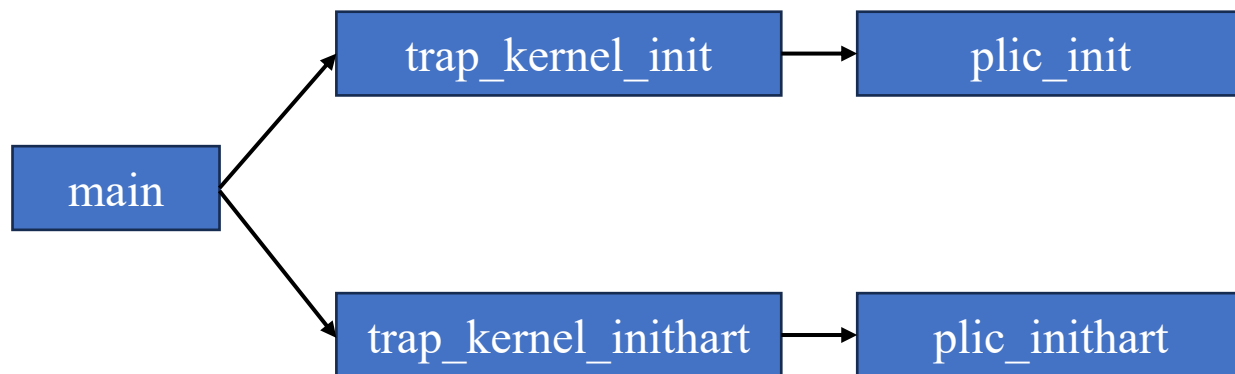
VS

中断模式

正常生活，耐心等待
成绩发布后会收到短信通知

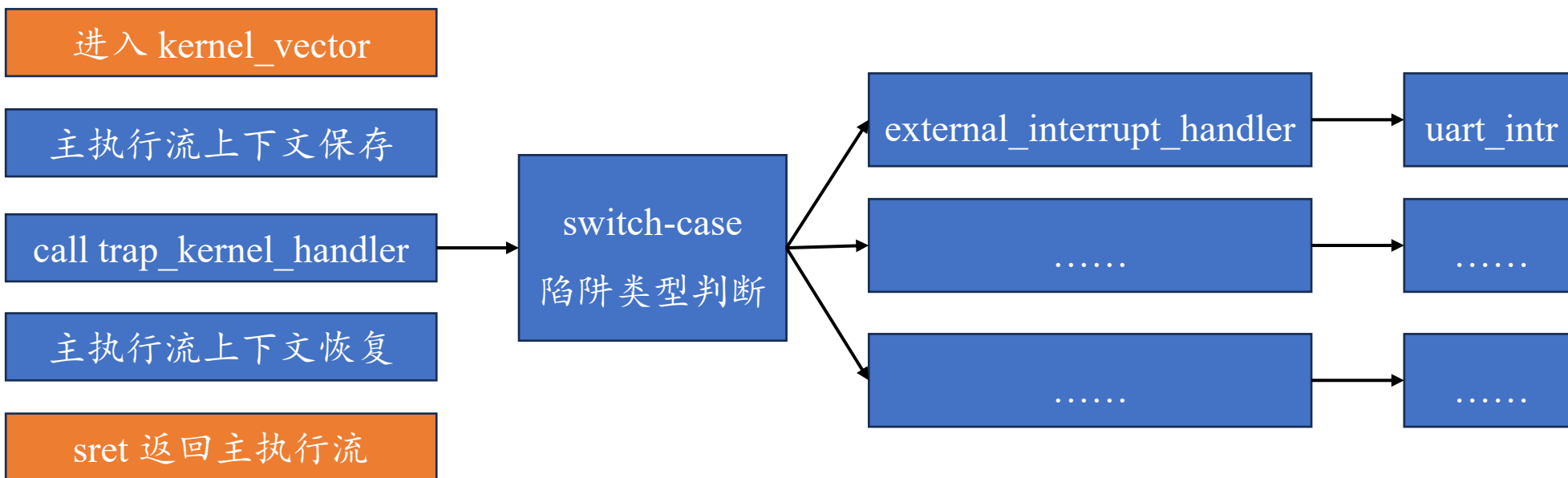
2. 串口中断

中断
感知



- 设置中断优先级和响应阈值
- 设置S-mode中断处理入口
- 使能各个中断开关
- 开启中断

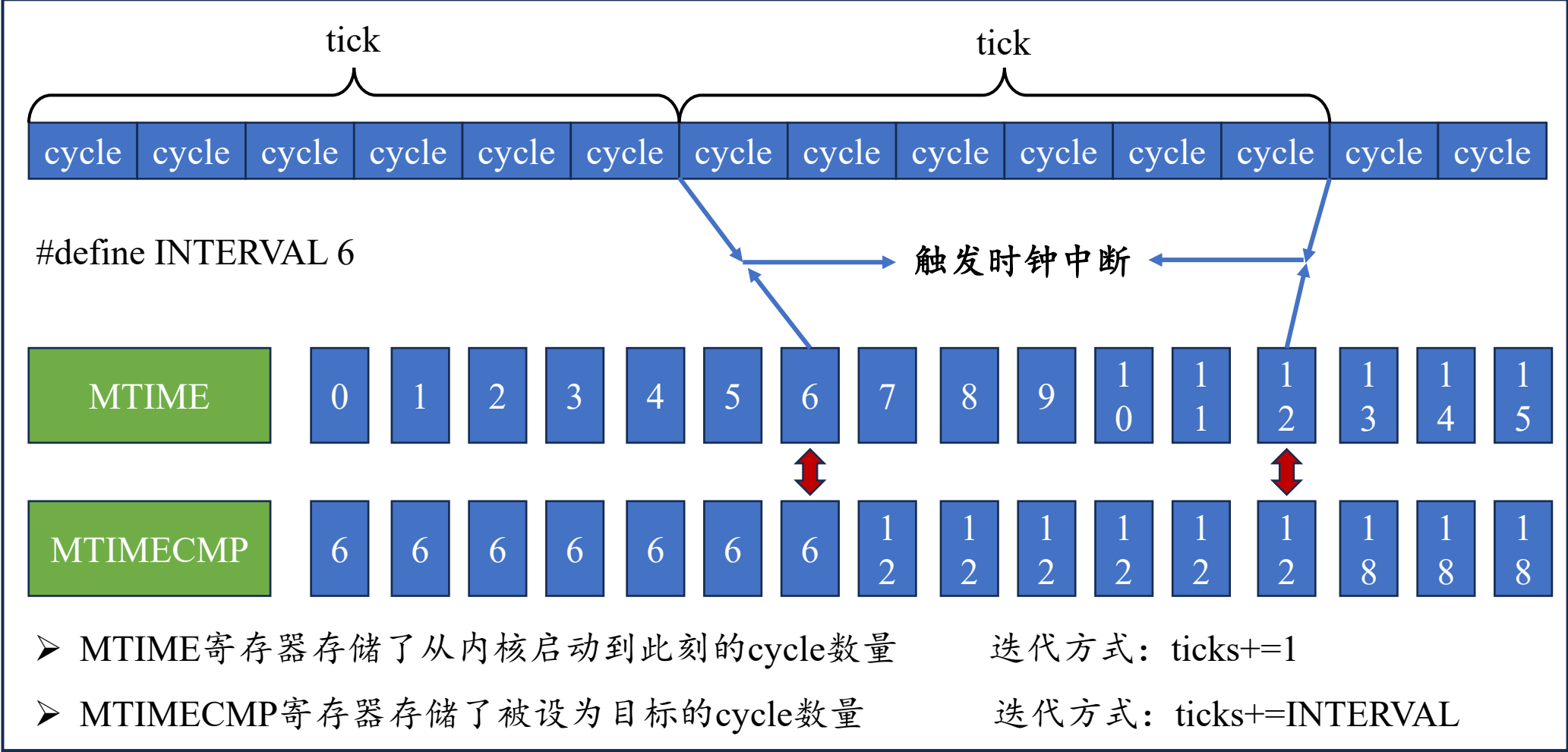
执行流
切换



3. 时钟中断

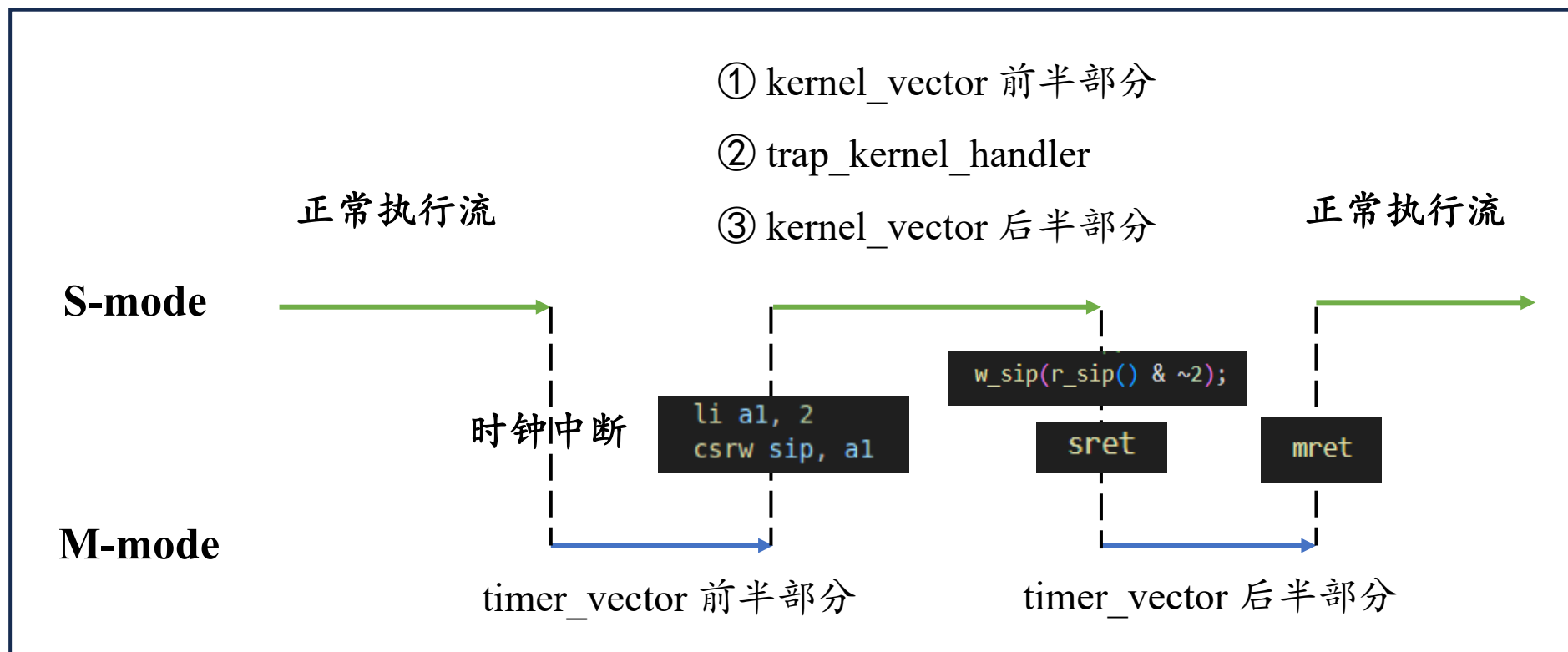
时钟是计算机的核心底层机制之一，是机器指令有序执行的“心跳”或“节拍”

RISC-V提供了以下时钟模型：



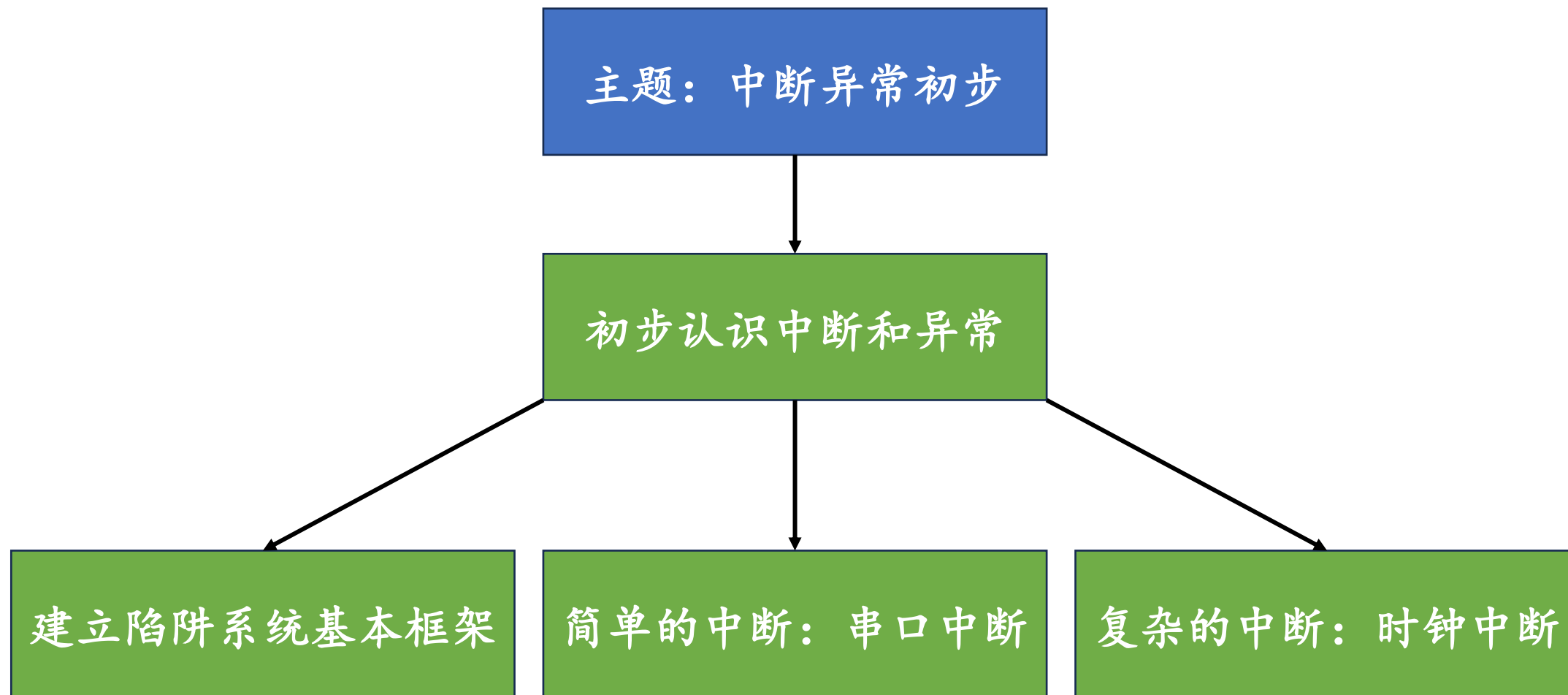
3. 时钟中断

- 相比串口中断,时钟中断的一个重要区别是: **相关寄存器只能在M-mode访问**
- 因此,时钟中断处理分为M-mode和S-mode两个逻辑部分



- OS内核的时钟中断处理由**M-mode时钟中断**和**S-mode软件中断**共同构成
- 前者负责MTIMECMP寄存器更新,后者执行OS内核的时钟相关逻辑

4. 实验小结



打断现有执行流的能力是OS内核掌控用户进程和外设能力的关键

一阶段的三个实验构建了OS内核的基础设施，为二阶段进程的诞生完成铺垫